Design and Bit-Exact Verification of a Real-Time Digital Microphone Filter on FPGA and SKY130

Saman Fuyal
Department of Electrical and Electronics Engineering
Kathmandu University
Dhulikhel, Nepal
smnfyl@gmail.com

***Abstract*—Digital MEMS microphones deliver audio as a one-bit pulse-density-modulated (PDM) stream, so every system that uses them needs a decimation and filtering chain. This paper presents such a chain on a Nexys A7-50T board and the method used to verify it. A fourth-order cascaded integrator-comb (CIC) decimator and a 63-tap droop-compensating FIR filter turn the 2.5 MHz PDM stream into 24-bit samples at 50 kHz. A DC blocker follows, then one of five filters chosen at run time: four 511-tap linear-phase filters that share one DSP48E1 multiplier, and a 31-tap parallel filter. Storing each coefficient bank with its own power-of-two scale recovers up to 14.2 dB of the stopband attenuation that 18-bit quantization would cost. A Python model reproduces every rounding and saturation step, and the RTL matched it sample for sample in block, chain and pin-level simulation; the pin-level test also exposed a byte-loss defect that the block tests missed. A UART port repeats the comparison on the board. On the XC7A50T the design meets 100 MHz with 1.462 ns of setup slack. The 31-tap filter enters the open-source LibreLane flow for the SkyWater 130 nm process unchanged, where pipelining raises the routed clock from 64 MHz to 166 MHz for 31% more cell area.**

Keywords*—FPGA, PDM microphone, CIC decimator, FIR filter, fixed-point arithmetic, bit-exact verification, hardware-in-the-loop, SKY130, LibreLane

I. Introduction

Digital MEMS microphones are found in phones, laptops, hearing aids and voice assistants. Each one contains a delta-sigma modulator and outputs a one-bit stream at a few megahertz [1]. Turning that stream into audio takes low-pass filtering and decimation, usually a CIC stage followed by one or more FIR stages [2], [3]. Software-defined radios use the same structure in their receive path [4].

On FPGA teaching boards the microphone is often read with a counter that adds up the ones in a fixed window; the Nexys A7 reference manual suggests 128 bits per output sample [5]. Such a counter is a single moving-average stage. It lowers the rate, but its sinc response folds much of the modulator noise back into the audio band and bends the passband. Published FPGA microphone systems build the chain from vendor IP or generated HDL and report resources and clock speed [6], [7]. They say less about whether the hardware output matches the design model exactly, or how a filter designed for an FPGA can be carried towards an integrated circuit.

This work addresses those points with free tools on a low-cost board. Its contributions are:

- a complete PDM-to-audio chain on the Nexys A7-50T in one 100 MHz clock domain, with five filters selectable at run time, four of which share one multiplier;
- per-bank power-of-two coefficient scaling, which raises the stopband attenuation of the narrow filters by 6.6 to 14.2 dB for the cost of a four-way shift selection;
- a verification flow in which a Python model, the RTL at three levels of integration and the board itself are compared bit for bit, the last through a UART hardware-in-the-loop port;
- a 31-tap filter core written once and used both on the FPGA and in the open-source SKY130 ASIC flow, in pipelined and unpipelined forms.

Section II reviews related work, Sections III and IV describe the architecture and design, Section V the verification, Section VI the results, and Section VII concludes.

II. Related Work and Research Gap

Hogenauer introduced the CIC filter as a multiplier-free decimator [2]. Its passband droop is usually corrected by a short FIR filter, and Kwentus et al. showed that sharpening improves both the droop and the alias rejection [8]. Textbooks treat this cascade as standard practice for oversampled converters [3], [4].

Todorović et al. built an FPGA platform for a MEMS microphone array on a Spartan-6 device [6]. A fifth-order CIC, two half-band stages and a low-pass FIR filter, all Xilinx IP cores generated in System Generator, decimate a 3.072 MHz PDM stream by 64 to 48 kHz. The chain was not tested on its own; the authors compare measurements of the whole beamforming system with Matlab simulations. Da Silva et al. reviewed FPGA beamformers that use digital MEMS arrays and describe PDM demodulation and filtering as their common front end [9]. Ilies et al. compared three CIC implementations for a 3.072 MHz MEMS microphone on an Artix-7 device, generated by HDL Coder, by the AMD CIC Compiler and from open-source RTL [7]. They report LUT, register and clock-speed differences, and the three outputs agree to within about 1.6 units on a signal of roughly 4700 units peak to peak, which the authors call functionally equivalent rather than bit-exact.

On the silicon side, OpenLANE and its successor LibreLane combine open-source tools into a complete RTL-to-GDSII flow [10], [11], the SkyWater SKY130 process design kit is freely available [12], and shared shuttles such as Tiny Tapeout let students fabricate small designs [13].

Table I compares these works. The FPGA designs treat the decimator either as a resource and speed problem or as the front end of a larger system. None of them ties the word lengths to the stopband the filters achieve, compares the hardware output with a reference model on the board itself, or

carries a filter core unchanged from the FPGA into an ASIC flow. The present work does all three.

TABLE I. COMPARISON WITH RELATED FPGA MICROPHONE FRONT ENDS

Work	Device	Decimation chain	Filters	Verification	ASIC
[5]	Artix-7	count ones, 128 bits	1 fixed	not reported	no
[6]	Spartan -6	CIC5, 2 half-band, FIR (IP)	1 fixed	system vs. Matlab	no
[7]	Artix-7	CIC only, 3 versions	–	within 1.6 units	no
This work	Artix-7	CIC4, 63-tap comp., DC block	5 (switch)	bit-exact: model, RTL, board	same RTL

III. SYSTEM ARCHITECTURE

Fig. 1 shows the signal path. The board provides a 100 MHz oscillator, an Analog Devices ADMP421 PDM microphone [14] and a mono audio output whose open-drain input feeds a fourth-order Sallen-Key Butterworth low-pass filter [5]. All logic runs on the 100 MHz clock. The microphone clock is a registered data output, so the design has no derived clock, and its only clock-domain crossings are two-flop synchronizers on the asynchronous inputs (switches, button, UART receive and microphone data). The CIC registers are 21 bits wide. The filters carry 24-bit data with 18-bit coefficients and a 48-bit accumulator, and the output is 16 bits at 50 kHz.

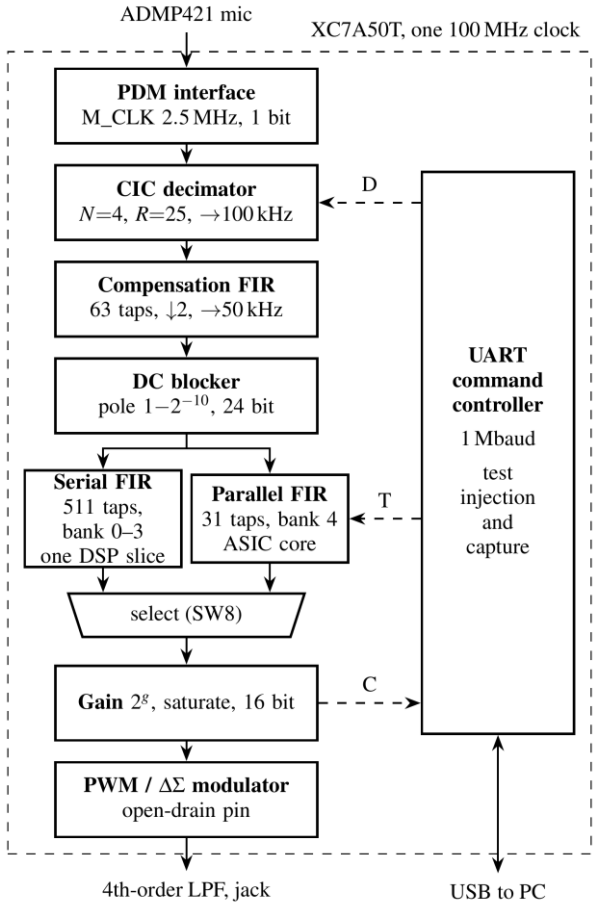


Fig. 1. Signal chain of the audio filter. Solid lines carry live audio. Dashed lines are the test paths of the UART controller: D injects PDM bits, T injects filter input samples and C captures the output.

IV. DESIGN METHODOLOGY

A. Design Flow and Coding Rules

Every constant (rates, widths, filter edges, shifts) is defined once in a Python file. Scripts design and quantize the filters, write the coefficient ROMs as Verilog case statements and generate the test vectors, so the RTL, the model and the tests share one source. The RTL is plain Verilog-2001 written to four rules: logic modules contain no initial blocks, delays or unbounded loops; every combinational block assigns a default to each output, so no latch can be inferred; combinational logic (blocking assignments) and registers (non-blocking assignments) sit in separate blocks; and each file is divided into ports, declarations, combinational logic and sequential logic. Verilator lint with all warnings enabled [15] runs clean, and Yosys [16] reports no inferred latch.

B. PDM Interface and Sampling Phase

The microphone clock M_CLK is the 100 MHz clock divided by 40, or 2.5 MHz, inside the ADMP421 range of 1 MHz to 3.3 MHz [5]. With its left/right select pin low, the microphone drives DATA during one half of the clock period and releases the line during the other half [5], [14], so the sample must fall inside the driven half. DATA passes through a two-flop synchronizer and is taken at count 38 of 40, about 50 ns before the falling edge at which the microphone releases the line (Fig. 2a). Switch SW6 moves the sampling point to the other half period. Sampling the released line returns noise, so the switch also checks the timing on the board. The top-level testbench models the released half as high impedance, and a wrong phase would then fail the bit-exact comparison.

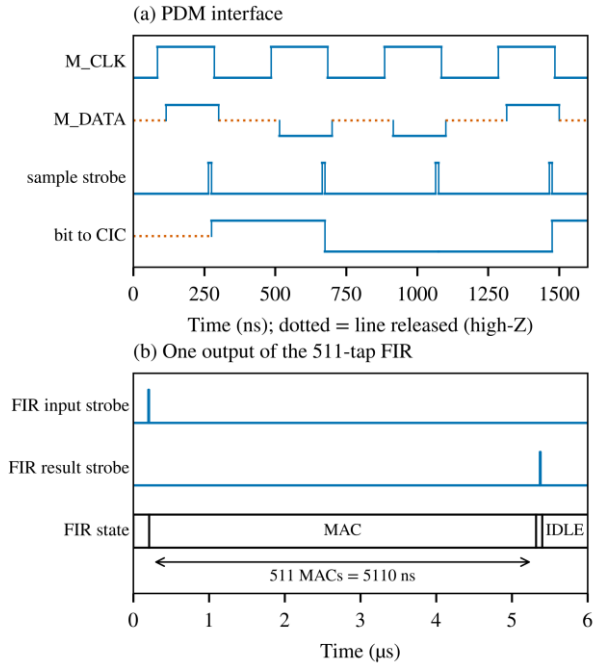


Fig. 2. Simulated waveforms (Icarus Verilog). (a) Microphone interface: the line is released (dotted) in the half period the microphone does not drive. (b) One output of the 511-tap filter.

C. CIC Decimator

A CIC filter with N = 4 stages, decimation R = 25 and differential delay M = 1 reduces the rate to 100 kHz without multipliers [2]. Its transfer function and DC gain are

$$H(z) = \left[\frac{1 - z^{-R}}{1 - z^{-1}}\right]^{N}, \quad H(1) = R^{N} = 390\,625 \tag{1}$$

The PDM bit is mapped to ±1, so the registers need 2 + ⌈N $\log_2$(RM)⌉ = 21 bits. The integrators may overflow: two's-complement arithmetic is modular, and the comb outputs stay exact as long as the registers have that width. The CIC output is shifted left by three bits, which places its full scale at 0.37 of the 24-bit range and leaves headroom for the passband boost of the next filter.

D. Droop Compensation and Second Decimation

The CIC response falls by 1.47 dB at 16 kHz. A 63-tap linear-phase FIR filter at 100 kHz, which also decimates by two, corrects the droop and removes the band that would alias into the audio band [8]. Its coefficients come from a weighted least-squares fit to 1/|H(f)| from 0 to 16 kHz, with a stopband from 25 kHz. After quantization the cascade is flat to within 0.006 dB up to 16 kHz and attenuates the 25 kHz to 50 kHz band by 95.1 dB (Fig. 3).

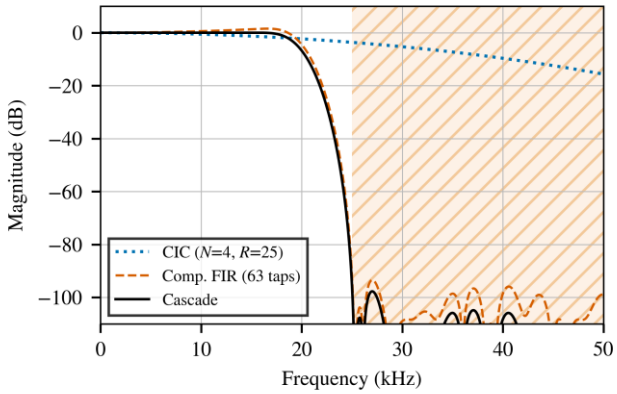


Fig. 3. CIC, compensation filter and their cascade at the 100 kHz rate. The hatched band aliases after decimation by two and is attenuated by 95.1 dB.

E. DC Blocker

MEMS microphones and the CIC produce a DC offset that the gain stage would otherwise amplify into saturation. A first-order high-pass without a multiplier removes it:

$$y[n] = x[n] - \lfloor a[n]/2^{K} \rfloor, \quad a[n+1] = a[n] + y[n] \tag{2}$$

With K = 10 the pole lies at $1 - 2^{-10}$ and the corner frequency near 7.8 Hz.

F. Selectable Filter Banks and Coefficient Scaling

Four 511-tap linear-phase filters were designed with the Parks-McClellan algorithm [17]: a pure delay used as bypass, a 1 kHz low-pass, a 2 kHz high-pass and a 0.4 kHz to 3.4 kHz band-pass based on the telephone voice band [18]. All four have a group delay of 255 samples (5.1 ms), so switching banks does not shift the signal in time. Coefficients use 18 bits to match the DSP48E1 multiplier [19].

Narrow-band filters have small coefficients; the largest low-pass tap is 0.047. In a fixed Q2.16 format the top five bits of every coefficient are then unused, and the quantization floor limits the stopband to about 70 dB. Each bank b is therefore stored multiplied by 2^{s}, where $s = s_b$ is the largest exponent for which the coefficients still fit in 18 bits, and the accumulator is shifted right by $S = 16 + s_b$:

$$y[n] = \mathrm{sat}\left(\left(\sum_{k=0}^{510} h_b[k]\, x[n-k] + 2^{S-1} \right) \gg S \right) \tag{3}$$

Table II shows the effect: the stopband of the low-pass filter rises from 70.1 dB to 84.3 dB, within 1 dB of the unquantized design. Fig. 4 shows the quantized responses together with the 31-tap filter of Section IV-H.

TABLE II. FILTER BANKS OF THE 511-TAP FIR (F_s = 50 KHZ)

Bank	Edges (kHz)	s_b	Float (dB)	Q2.16 (dB)	Scaled (dB)	Ripple (dB)
0 bypass	–	0	–	–	–	0
1 low-pass	1.0 / 1.4	5	85.2	70.1	84.3	0.028
2 high-pass	1.6 / 2.0	1	88.0	70.6	77.2	0.023
3 band-pass	0.4 to 3.4	3	83.2	69.3	79.3	0.035

[a] Float, Q2.16 and scaled give the stopband attenuation of the unquantized design, of 18-bit coefficients without scaling, and of the scaled coefficients used in hardware.

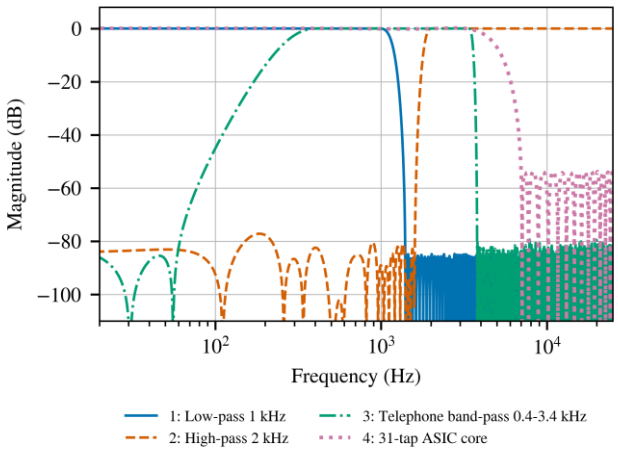


Fig. 4. Magnitude responses of the quantized filter banks 1 to 3 and of the 31-tap parallel filter (bank 4).

G. Time-Multiplexed FIR Architecture

At 100 MHz and 50 kHz there are 2000 clock cycles per output sample, so the 511-tap filter computes one product per cycle (Fig. 5). Samples are kept in a 512-word circular buffer in simple-dual-port block RAM, and coefficients in a 4 × 512 ROM addressed by bank and tap. The pipeline of RAM and ROM read, operand registers, multiply, accumulate, rounding, and shift with saturation maps onto the A, B, M and P registers of a DSP48E1. One output takes 511 + 6 = 517 cycles, about 26% of the budget (Fig. 2b). The same module, with decimation by two and a single bank, implements the compensation filter. An input that arrives while the filter is busy raises an overrun flag that lights a red LED, so a lost sample is always visible. After reset the delay line is cleared, which gives hardware and model the same starting state.

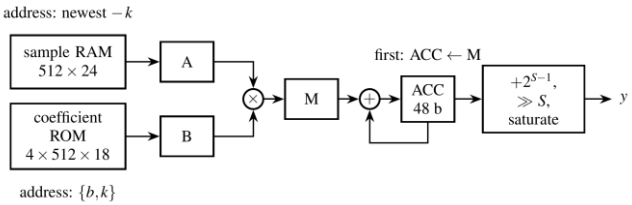


Fig. 5. Serial multiply-accumulate datapath of the 511-tap filter. The tap index k runs from 0 to 510 for each output; flags that travel with the data mark the first and last products.

H. Parallel FIR Core Shared with the ASIC

Bank 4 is a 31-tap linear-phase low-pass filter with 16-bit data and coefficients and a 53.7 dB stopband. It computes all

taps in parallel, which is the form an ASIC without RAM macros needs. Symmetry halves the multipliers to 16 through pre-adders, and because the coefficients are constants, ASIC synthesis reduces each multiplier to shifts and additions (67 non-zero canonical-signed-digit terms in total) [20]. A parameter selects one of two micro-architectures [21]: PIPE = 0 registers only the input shift register and the output, while PIPE = 1 adds a register after the pre-adders, the multipliers and every adder-tree level. The FPGA uses PIPE = 1, and switch SW8 routes the audio through this core instead of the 511-tap filter. The same Verilog file is the input of the ASIC flow in Section VI-D.

I. Gain and Output Stage

A switch-selected gain of 2^g (g = 0 to 15, 0 dB to 90 dB) with saturation makes up for the low microphone level; speech at 60 dB SPL is roughly −60 dBFS at the ADMP421 output [14]. The top 16 bits drive either a 10-bit PWM modulator with a 97.7 kHz carrier, above the 50 kHz minimum that Digilent recommends for the board filter [5], or a first-order delta-sigma modulator running at 100 MHz [1]. The output pin is open drain, as the board requires, and the modulator input is held at mid-scale in test mode and while the amplifier is off.

J. Control and Hardware Test Port

A command controller (Fig. 6) receives commands over the board's USB-UART at 1 Mbaud. 'P' and 'S' return an identifier and the switch and error state. 'T' sends 24-bit samples through one filter bank and returns the results. 'D' sends PDM bytes through the whole chain and returns 16-bit audio samples. 'C' records 16 384 live output samples (0.33 s) and sends them to the host. 'T' and 'D' first clear the chain and mute the output, so the board reproduces the Python model exactly, and restore live operation when they finish. A watchdog returns the controller to IDLE when the host stops sending for 0.17 s.

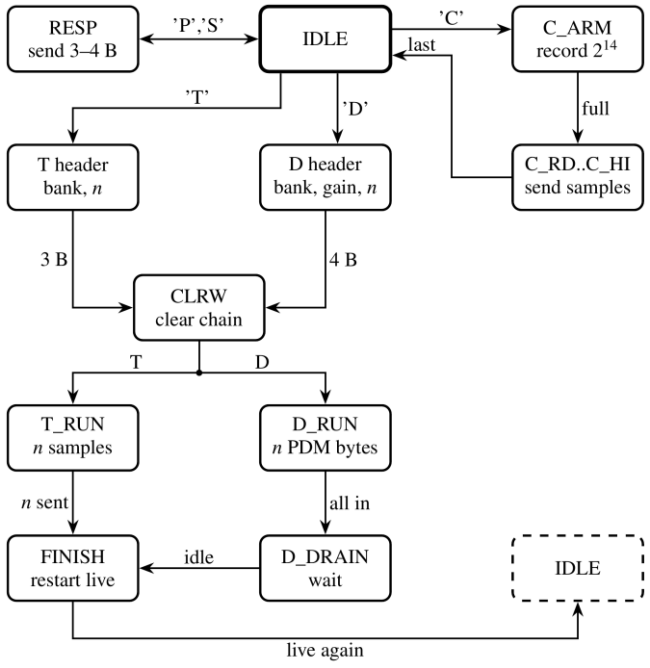


Fig. 6. State diagram of the UART command controller. CLRW clears the filter delay lines before a test command; a byte that arrives during the clear is held and processed afterwards. Header and run states return to IDLE if the host stops sending for 0.17 s.

K. Hardware Implementation

The pin constraints come from Digilent's master constraint file. Asynchronous inputs are declared as false paths because each passes through a synchronizer, and the microphone data timing is set by the sampling phase of Section IV-B. Switches select the bank, gain, sampling half and modulator; the LEDs and the seven-segment display show the signal level, the settings and a sticky error flag. The RTL hierarchy that Vivado 2018.2 elaborates matches Fig. 1 block for block, and the DC blocker elaborates to a shift, a subtractor, an adder and two comparators, with no multiplier.

V. Verification Methodology

A. Golden Model and Testbenches

The Python model reproduces every register-level detail of the RTL: CIC wrap-around and decimation phase, FIR rounding by adding 2^{S-1} before an arithmetic shift, saturation, DC-blocker truncation and the gain stage. Because the model also writes the coefficient ROMs and the test vectors, RTL and model share one source of coefficients. Table III lists the self-checking testbenches, which run in Icarus Verilog [22]. Their stimuli include impulses, full-scale random data, sign patterns that drive the accumulator into saturation, and PDM streams from a second-order delta-sigma modulator model. The chain testbench supplies PDM bits 2.5 times faster than the microphone to stress the pipeline; no overrun occurred. A separate plain-Verilog testbench targets the Vivado 2018.2 simulator.

TABLE III. Verification Summary (Simulation)

Testbench	Scope	Samples	Result
tb_cic	CIC: random, tones, DC	624	PASS
tb_fir	4 banks, saturation, overrun; comp. FIR	12 268	PASS
tb_chain	PDM to audio, 5 settings; DC blocker	11 500	PASS
tb_top	pins: microphone model, UART P/S/T/D/C (full FIFO), watchdog, open drain	702	PASS
tb_fir_par	31-tap core, PIPE = 0 and 1	4 224	PASS

B. Defects Found at the Top Level and on the Board

Every block passed its own test, yet the first pin-level run of the 'T' command returned 39 of 40 samples, all wrong. At the start of a command the controller clears the filter delay line and then waits; that wait (11 µs) was longer than one UART byte (10 µs), so the first data byte was dropped and every later sample was misaligned by one byte. The fix shortens the wait to 6 µs, still longer than the 512-cycle clear, and adds a one-byte holding register so that no byte can be lost. The defect lay in the interaction between the protocol timing and the clear sequence, which no block-level test exercises.

A second defect appeared only on the board: the capture command returned 16 641 of 32 768 bytes. The controller registers its write to the UART transmit FIFO, so a byte enters the FIFO one clock after the controller has checked the full flag. The capture dump writes the two bytes of a sample on consecutive clocks. With one slot free, both writes passed the check and the FIFO dropped the second. Once the 512-byte FIFO had filled, every second byte was lost, which predicts 16 640 bytes. The 'T' and 'D' replies never fill the FIFO because their output is paced by their input, and the top-level test captured only 128 bytes. An almost-full flag now stops the

writer one slot early, and the top-level test captures 2048 bytes and compares each one with the samples that were recorded.

C. Board-Level Comparison

On the board the host repeats the comparison through the UART port. It sends an impulse and random full-scale blocks through each bank, and PDM streams through the whole chain, and compares every returned sample with the model. The host tool also runs against a software copy of the board; in that mode it compared 67 920 samples with no mismatch before the hardware was available.

VI. Results and Discussion

A. Simulation Results

Fig. 7 overlays the RTL output and the model for the band-pass setting; the difference is zero for every sample, and the same holds for all testbenches in Table III. The 511-tap filter delivers a result 517 cycles after its input, and the whole chain has a group delay of 5.43 ms.

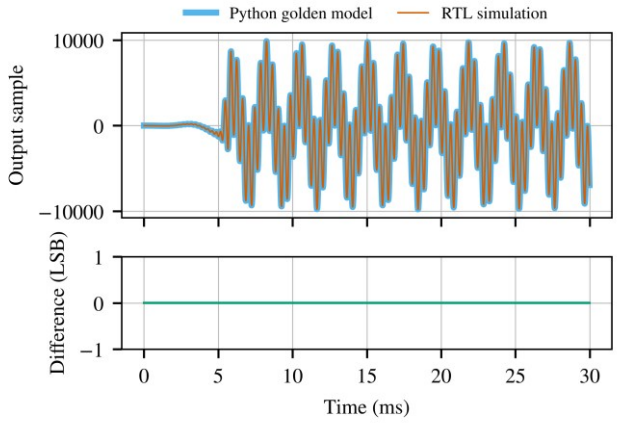


Fig. 7. Full-chain output for bank 3: RTL simulation against the Python model. The difference is zero over all 1500 samples.

B. FPGA Implementation

Table IV compares the Yosys synth_xilinx estimate [16] with Vivado 2018.2 after placement and routing on the XC7A50T. The design meets the 100 MHz constraint with a worst setup slack of 1.462 ns, which corresponds to about 117.1 MHz, and a hold slack of 0.048 ns. Each serial filter uses one DSP slice, the 16 constant multipliers of the parallel core use most of the others, and eight of the block-RAM tiles hold the capture buffer.

TABLE IV. FPGA Implementation on the XC7A50T

Resource	Yosys estimate	Vivado	Available
LUT	2,173	1,948	32 600
Flip-flop	2,274	2,191	65 200
DSP48E1	18	18	120
Block RAM (36 Kb tiles)	10.5	10.5	75
WNS / WHS at 100 MHz (ns)	–	1.462 / 0.048	–
On-chip power (mW)	–	95	–

C. On-Board Measurements

The hardware-in-the-loop self-test sent 17,920 samples through the five filters and 50,000 samples through the whole chain. Every output matched the golden model bit for bit (0 mismatches in either test), both over JTAG and after the board had booted from flash. Fig. 8 shows bank 3 measured with 60 stepped sines through the 'T' path; above −60 dB the four MAC banks agree with the quantized design to 0.001 dB, and the 31-tap core to 0.03 dB, the difference coming from its 16-bit data path.

For the live path, a pure 1 kHz sine from a laptop speaker 10 cm from the microphone gave an SNR of 42.2 dB with the harmonics excluded, a THD of −37.6 dB and a SINAD of 36.3 dB at −32 dBFS. These values include the speaker and the room. A phone speaker at the same level gave a THD of −14.7 dB that improved by 16 dB when its volume was lowered, so that distortion was acoustic. In the same setup the low-pass bank removed a 3 kHz tone by 85.5 dB and the SKY130 core removed an 8 kHz tone by 56.7 dB, against 89.4 and 57.2 dB by design; the first figure is limited by the noise floor of a 16 384-sample capture. Both PDM sampling phases (SW6) gave the same result within 1 dB.

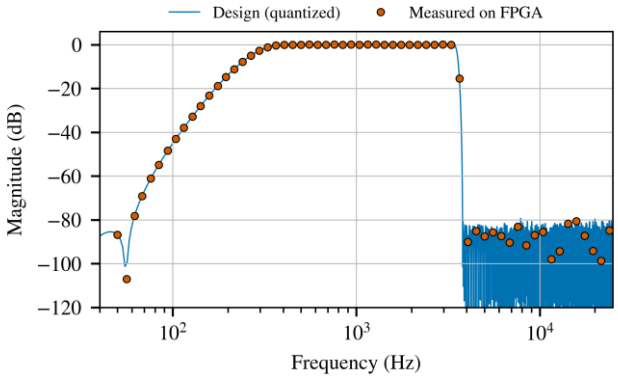


Fig. 8. Bank 3 (telephone band-pass): designed response and the response measured on the FPGA with 60 stepped sines through the UART test path.

D. ASIC Implementation

Table V gives a pre-layout estimate from Yosys with ABC mapping to the sky130_fd_sc_hd library at the typical corner (25 °C, 1.8 V). The delay counts gate delays only, without wires, clock-to-output and setup time. Pipelining cuts the critical gate delay from 14.38 ns to 5.74 ns for about 17% more cell area, mostly from the extra 1,677 versus 514 flip-flops. The LibreLane runs add placement, clock-tree synthesis, routing, static timing with extracted parasitics, and DRC and LVS checks. Each version is routed at a clock it meets, its shortest period is taken as that clock minus the worst setup slack at the typical corner, and a second run at that period confirms it. The slow corner (100 °C, 1.6 V) is not closed at these periods; a product would need a longer clock or more pipelining there. Fig. 9 shows the routed layout of the pipelined core.

TABLE V. SKY130 Results for the 31-Tap Core

Metric	PIPE = 0	PIPE = 1
Cells (pre-layout)	10,393	11,862
Flip-flops	514	1,677
Cell area (µm^2)	85,840	100,199
Critical gate delay (ns)	14.38	5.74
Min. period after routing (ns)	15.6	6.0
Area after routing (µm^2)	145,592	190,643
Power (mW)	10.77	27.45
DRC / LVS errors	0 / 0	0 / 0

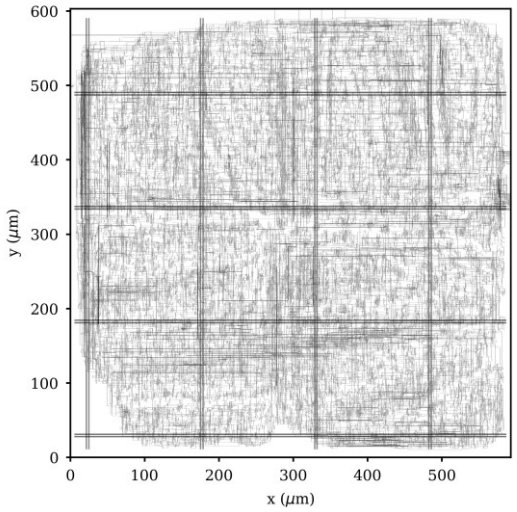


Fig. 9. Routed layout of the pipelined 31-tap core on SKY130.

E. Discussion and Limitations

The design trades latency for hardware. The 511-tap filters add 5.1 ms of delay and the whole chain 5.43 ms, which suits monitoring and demonstration but would call for shorter or minimum-phase filters in a hearing aid. On this board the microphone data line keeps its last value while released, so both PDM sampling phases read valid bits and SW6 cannot catch a wrong phase; a board without that behavior would need the phase checked again. The live-path numbers are bounded by the acoustic source and the room rather than by the filter, and the board's output filter and PWM carrier limit the analog quality at the jack, so quality is measured digitally through the capture command.

VII. Conclusion and Future Work

A digital microphone filter was designed for the Nexys A7-50T with five run-time selectable filters, four of them on one DSP slice, and per-bank coefficient scaling that recovers up to 14.2 dB of stopband attenuation. A bit-exact verification flow covers every block, the full chain and the pin-level top level, and extends to the board through a UART test port. The 31-tap core runs on the FPGA and enters the open-source SKY130 flow from the same source file. Future work includes a minimum-phase mode with lower latency, an adaptive noise-cancellation stage, and fabrication of the core on a shared shuttle.

Acknowledgment

The author thanks the Department of Electrical and Electronics Engineering, Kathmandu University, for access to the Nexys A7 board.

References

[1] R. Schreier and G. C. Temes, *Understanding Delta-Sigma Data Converters*. Piscataway, NJ, USA: Wiley-IEEE Press, 2005.

[2] E. B. Hogenauer, “An economical class of digital filters for decimation and interpolation,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 29, no. 2, pp. 155–162, Apr. 1981.

[3] R. G. Lyons, *Understanding Digital Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[4] U. Meyer-Baese, *Digital Signal Processing with Field Programmable Gate Arrays*, 4th ed. Berlin, Germany: Springer, 2014.

[5] Digilent Inc., “Nexys A7 reference manual.” [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual

[6] D. Todorović, I. Salom, V. Čelebić, and J. Prezelj, “Implementation and application of FPGA platform with digital MEMS microphone array,” in *Proc. 4th Int. Conf. Electrical, Electronics and Computing Engineering (IcETRAN)*, Kladovo, Serbia, Jun. 2017.

[7] E. Ilies, M. Marinca, and A. Gontean, “Post-implementation evaluation of CIC filters for digital audio applications on FPGA,” *J. Low Power Electron. Appl.*, vol. 16, no. 1, Art. no. 5, 2026, doi: 10.3390/jlpea16010005.

[8] A. Y. Kwentus, Z. Jiang, and A. N. Willson, “Application of filter sharpening to cascaded integrator-comb decimation filters,” *IEEE Trans. Signal Process.*, vol. 45, no. 2, pp. 457–467, Feb. 1997.

[9] B. da Silva, A. Braeken, and A. Touhafi, “FPGA-based architectures for acoustic beamforming with microphone arrays: Trends, challenges and research opportunities,” *Computers*, vol. 7, no. 3, Art. no. 41, 2018, doi: 10.3390/computers7030041.

[10] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based tapeout-proven flow,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2020, pp. 1–6.

[11] The LibreLane Contributors, “LibreLane: An open-source ASIC implementation flow.” [Online]. Available: https://github.com/librelane/librelane

[12] SkyWater Technology and Google, “SKY130 open-source process design kit,” 2020. [Online]. Available: https://github.com/google/skywater-pdk

[13] M. Venn, “Tiny Tapeout: A shared silicon tapeout platform accessible to everyone,” *IEEE Solid-State Circuits Mag.*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10584359

[14] Analog Devices, “ADMP421: Low-profile, low-power, bottom-port digital output MEMS microphone,” data sheet.

[15] W. Snyder, “Verilator.” [Online]. Available: https://www.veripool.org/verilator/

[16] C. Wolf, “Yosys Open SYnthesis Suite.” [Online]. Available: https://github.com/YosysHQ/yosys

[17] J. H. McClellan, T. W. Parks, and L. R. Rabiner, “A computer program for designing optimum FIR linear phase digital filters,” *IEEE Trans. Audio Electroacoust.*, vol. 21, no. 6, pp. 506–526, Dec. 1973.

[18] ITU-T, “Transmission performance characteristics of pulse code modulation channels,” Recommendation G.712, 2001.

[19] AMD Xilinx, “7 Series DSP48E1 slice user guide (UG479).”

[20] R. I. Hartley, “Subexpression sharing in filters using canonic signed digit multipliers,” *IEEE Trans. Circuits Syst. II*, vol. 43, no. 10, pp. 677–688, Oct. 1996.

[21] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. New York, NY, USA: Wiley, 1999.

[22] S. Williams, “Icarus Verilog.” [Online]. Available: https://github.com/steveicarus/iverilog